

به نام خدا

گزارش تمرین کامپیوتری دوم

سیگنال‌ها و سیستم‌ها-دکتر اخوان

امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

بخش اول:

-۱

```
[file,path]=uigetfile({'*.jpg;*.bmp;*.png;*.tif'},'Choose an image');  
s=[path,file];  
img = imread(s);
```

uigetfile: یک پنجره باز می‌کند تا کاربر بتواند فایلی با فرمت‌های مشخص (مثل *JPG* و *PNG*) را انتخاب کند.

s = [path, file]: مسیر کامل فایل را می‌سازد.

imread(s): تصویر را از این مسیر می‌خواند و در متغیر *img* ذخیره می‌کند.

=۲

```
img = imread(s);  
gSizeIMG = imresize(img,[300,500]);
```

-۳

```
function y = mygrayfun(x)
```

```
    if size(x, 3) == 3  
        y = x(:,:,1)*0.299+ x(:,:,2)*0.578+ x(:,:,3)*0.114;  
    else  
        y = x;  
    end  
end
```

این فانکشن دو کار اصلی را انجام می‌دهد:

چک کردن تعداد کانال‌ها:

$\text{size}(x, 3) == 3$ بررسی می‌کند که آیا تصویر سه‌بعدی (رنگی) است یا نه). تصویر رنگی معمولاً سه کانال *RGB* دارد).

تبدیل به تصویر خاکستری:

اگر تصویر رنگی باشد، از فرمول وزنی برای تبدیل *RGB* به ابی استفاده می‌کند. مقادیر ۰.۲۹۹، ۰.۵۷۸، ۰.۱۱۴ وزن‌های هر کانال رنگی نسبت به خاکستری هستند. (همونطور که خود سوال خواسته)
این باعث میشه که تصویر خاکستری بسازد. و به عنوان خروجی فانکشن برمی‌گردد.

-۴

```

120 function y = mybinaryfun(x)
121     y=x;
122     for i = 1:size(y,1)
123         for j = 1:size(y,2)
124             if y(i,j,1)>=100
125                 y(i,j,1)=0;
126             else
127                 y(i,j,1)=1;
128             end
129         end
130     end
131 end

```

بررسی عملکرد این فانکشن:

تصویر ورودی λ به متغیر λ کپی می‌شود.

با دو حلقه تو در تو، هر پیکسل تصویر مرور می‌شود. حلقه‌ها هر سطر و هر ستون تصویر رو پیمایش می‌کنند.

اگر مقدار شدت رنگ پیکسل (اولین کانال) بیشتر یا برابر با ۱۰۰ باشد، این پیکسل به ۰ (سفید) تبدیل می‌شود.

در غیر این صورت، به ۱ (سیاه) تبدیل می‌شود.

-۵

```

function pic=myremovecom(picture,n)

[row,column]=find(picture==1);
POINTS=[row';column'];
POINTS_NUM=size(POINTS,2);
pic2=picture;
while POINTS_NUM>0
    point_ini=POINTS(:,1);
    POINTS(:,1)=[];
    %
    POINTS_NUM=size(POINTS,2);
    diff= abs(repmat(point_ini,1,POINTS_NUM)-POINTS);
    index=find(diff(1,:)<=1 & diff(2,:)<=1);
    newpoints=POINTS(:,index);
    POINTS(:,index)=[];

    current_obj=[point_ini newpoints];
    newpoints_len=size(newpoints,2);

    while newpoints_len>0
        POINTS_NUM=size(POINTS,2);
        diff= abs(repmat(newpoints(:,1),1,POINTS_NUM)-POINTS);
        index=find(diff(1,:)<=1 & diff(2,:)<=1);
        newpoints1=POINTS(:,index);
        POINTS(:,index)=[];

        newpoints(:,1)=[];
        newpoints=[newpoints newpoints1];
        current_obj=[current_obj newpoints1];
        newpoints_len=size(newpoints,2);
    end
    if size(current_obj,2) < n
        pics=pic2;
        ind=sub2ind(size(picture),current_obj(1,:),current_obj(2,:));
        pics(ind)=0;
        pic2=pics;
    end
    POINTS_NUM=size(POINTS,2);
end
pic=pic2;
end

```

هدف تابع:

تابع `myremovecom` برای حذف نواحی کوچک (با تعداد پیکسل کمتر از n) در یک تصویر باینری (تصویری که دارای مقادیر ۰ برای پس‌زمینه و ۱ برای شی است. (شی مد نظر ما همان حروف پلاک می‌باشد)) طراحی شده است. هدف اصلی، پاک‌سازی تصویر از نواحی غیرضروری و کوچک است تا فقط نواحی اصلی که نیاز داریم باقی بمانند.

بررسی عملکرد فانکشن:

شناسایی پیکسل‌های سفید:

با استفاده از `find(image == 1)`، مختصات (ردیف و ستون) همهٔ پیکسل‌های سفید (شیء) در تصویر شناسایی می‌شود.

این مختصات در یک ماتریس به نام `POINTS` ذخیره می‌شود.

حلقه اصلی:

در این حلقه، تابع نقاط سفید (نقاط موجود در `POINTS`) را یکی یکی پردازش می‌کند تا نواحی متصل به هم را شناسایی کند.

اگر هنوز نقاطی باقی مانده باشند (`POINTS_NUM > 0`)، نقطه‌ی اولیه از `POINTS` انتخاب و حذف می‌شود.

محاسبه فاصله و شناسایی نقاط نزدیک:

با استفاده از `repmat`، فاصله بین `point_ini` و باقی‌مانده نقاط محاسبه می‌شود.

نقاطی که فاصله آن‌ها کمتر از یا برابر ۱ است، به عنوان نقاط نزدیک به `point_ini` شناسایی و در متغیر `newpoints` ذخیره می‌شوند.

گسترش ناحیه:

سپس برای هر نقطه جدید، دوباره نقاط نزدیک به آن را شناسایی کرده و به `current_obj` (نواحی متصل) اضافه می‌کند.

این تکرار تا زمانی ادامه می‌یابد که دیگر نقاط جدیدی پیدا نشود.

حذف نواحی کوچک:

پس از شناسایی تمام نقاط متصل به یکدیگر:

اگر تعداد نقاط در `current_obj` کمتر از `n` باشد، این ناحیه به عنوان ناحیه‌ای کوچک در نظر گرفته می‌شود و از تصویر حذف می‌شود. این کار با تغییر مقادیر مربوط به این نواحی به ۰ انجام می‌شود.

در انتهای هر دور از حلقه، تعداد نقاط باقی‌مانده به روز می‌شود و حلقه ادامه می‌یابد تا زمانی که همه نقاط پردازش شوند.

پس از اتمام پردازش‌ها، تصویر تولیدشده (`pic2`) که شامل نواحی حذف‌شده است، به عنوان خروجی برمی‌گردد.

```

function [L,Ne]=mysegmentation(picture)
L=zeros(size(picture));
[row,column]=find(picture==1);
POINTS=[row';column'];
current_obj_num=1;
POINTS_NUM=size(POINTS,2);
while POINTS_NUM>0
    point_ini=POINTS(:,1);
    POINTS(:,1)=[];

    POINTS_NUM=size(POINTS,2);
    diff=abs(repmat(point_ini,1,POINTS_NUM)-POINTS);
    index=find(diff(1,:)<=1 & diff(2,:)<=1);
    newpoints=POINTS(:,index);
    POINTS(:,index)=[];

    current_obj=[point_ini newpoints];
    newpoints_len=size(newpoints,2);
    while newpoints_len>0
        POINTS_NUM=size(POINTS,2);
        diff=abs(repmat(newpoints(:,1),1,POINTS_NUM)-POINTS);
        index=find(diff(1,:)<=1 & diff(2,:)<=1);
        newpoints1=POINTS(:,index);
        POINTS(:,index)=[];
        newpoints(:,1)=[];

        newpoints=[newpoints newpoints1];
        current_obj=[current_obj newpoints1];
        newpoints_len=size(newpoints,2);
    end
    ind=sub2ind(size(picture),current_obj(1,:),current_obj(2,:));
    L(ind)=current_obj_num;
    current_obj_num=current_obj_num+1;
    POINTS_NUM=size(POINTS,2);
end
Ne=current_obj_num-1;
end

```

هدف

تابع mysegmentation برای شناسایی و برچسب‌گذاری نواحی متصل در یک تصویر باینری طراحی شده.

بررسی عملکرد فانکشن:

ایجاد ماتریس خالی:

$L = \text{zeros}(\text{size}(\text{picture}));$ یک ماتریس با همان ابعاد تصویر ورودی ایجاد می‌کند که در آن تمامی مقادیر به‌طور پیش‌فرض ۰ هستند. این ماتریس برای ذخیره لیبل‌های مختلف استفاده می‌شود.

شناسایی نقاط سفید:

$\text{find}(\text{picture} == 1)$ (ردیف و ستون) نقاط سفید (شیء) موجود در تصویر را شناسایی می‌کند و در متغیری به نام POINTS ذخیره می‌کند.

حلقه اصلی:

در این حلقه، تابع نقاط سفید (نقاط موجود در POINTS) را یکی یکی پردازش می‌کند تا نواحی متصل به هم را شناسایی کند.

شرط $\text{while POINTS_NUM} > 0$ اطمینان حاصل می‌کند که این فرآیند تا زمانی که نقاطی برای پردازش باقی مانده باشد، ادامه یابد.

انتخاب نقطه اولیه:

$\text{point_ini} = \text{POINTS}(:, 1);$ اولین نقطه در POINTS به‌عنوان نقطه اولیه انتخاب می‌شود و سپس از POINTS حذف می‌شود.

شناسایی نقاط نزدیک:

فاصله بین `point_ini` و سایر نقاط محاسبه شده و نقاطی که به این نقطه نزدیک هستند (منظور از نزدیکی همسایه بودن است) شناسایی و در متغیر `newpoints` ذخیره می‌شوند.

نقاط نزدیک به `point_ini` از `POINTS` حذف می‌شوند.

گسترش ناحیه:

یک حلقه داخلی برای گسترش ناحیه وجود دارد، که با استفاده از `newpoints`، دوباره نقاط نزدیک به آن‌ها شناسایی می‌شود و به `current_obj` اضافه می‌شود.

این حلقه تا زمانی که نقاط جدیدی پیدا نشود، ادامه می‌یابد.

برچسب‌گذاری ناحیه:

پس از شناسایی تمامی نقاط یک ناحیه متصل، با استفاده از `sub2ind`، اندیس‌های مربوط به این نقاط در ماتریس `L` به `current_obj_num` (شماره ناحیه فعلی) نسبت داده می‌شود.

سپس شمارنده `current_obj_num` یک واحد افزایش می‌یابد تا برای ناحیه بعدی آماده شود.

تعداد نواحی شناسایی شده:

در انتهای پردازش، تعداد نواحی شناسایی شده در متغیر `Ne` ذخیره می‌شود. این مقدار برابر با `current_obj_num - 1` است (شماره‌گذاری از ۱ شروع می‌شود).

-۷

ابتدا برای بارگذاری `trainset` که شامل حروف و اعداد پلاک انگلیسی برای کورلیشن گرفتن می‌باشد، یک فایل جدید به نام `mapset2trainingset` ایجاد می‌کنیم.

این کد تصاویر را از دایرکتوری `\p1\Map Set\` بارگذاری کرده و در یک آرایه به نام `TRAIN` ذخیره می‌کند. سپس، این آرایه شامل تصاویر و اولین حرف نام هر فایل را در فایل `TRAININGSET.mat` ذخیره می‌کند.

```

clc;clear;close all;clearvars;

directory = dir('./p1/Map Set/');
nam = {directory(3:end).name};
len = length(nam);

TRAIN = cell(2, len);

for i = 1:len
    currentFile = fullfile('./p1/Map Set/', nam{i});
    if isfile(currentFile)
        TRAIN(1, i) = {imread(currentFile)};
        TRAIN(2, i) = {nam{i}(1)};
    else
        warning('File does not exist: %s', currentFile);
    end
end

% Save the DATA
save('TRAININGSET.mat', 'TRAIN');

```

در ابتدا، کد یک مجموعه از تصاویر آموزشی (مثل حروف یا اعداد) را بارگذاری می‌کند. این تصاویر برای یادگیری مدل استفاده می‌شوند. سپس تعداد کل حروفی که در این مجموعه است را پیدا می‌کند و در `totalLetters` ذخیره می‌کند.

`final_output` یک آرایه خالی است که در آن حروف شناسایی شده ذخیره می‌شود. متغیر `t` به نظر می‌رسد در اینجا مورد استفاده قرار نگرفته باشد.

پس از آن کد شروع به بررسی هر ناحیه شناسایی شده در تصویر می‌کند. فرض کنید چند حرف یا علامت جدا از هم شناسایی شده‌اند (تعداد آن‌ها `Ne` است).

برای هر ناحیه:

کد موقعیت آن ناحیه (ردیف و ستون) را پیدا می‌کند. قسمتی از تصویر که مربوط به آن ناحیه است (حرف یا علامت) را برمی‌دارد و آن را به γ نسبت می‌دهد. تصویر γ را معکوس می‌کند و آن را نمایش می‌دهد. سپس تصویر را به ابعاد مشخصی (42×24) تغییر اندازه می‌دهد تا با حروف آموزشی یکسان شود و دوباره نمایش می‌دهد (تاخیر ۰.۲ ثانیه‌ای برای این است که تصویر به وضوح دیده شود).

محاسبه کورلیشن و شناسایی حروف بر اساس همبستگی

یک از حروف موجود در مجموعه آموزشی عمل می‌کند.

یک آرایه به نام `ro` برای ذخیره نتایج همبستگی ایجاد می‌کند. سپس برای هر حرف در مجموعه آموزشی (که در `TRAIN` ذخیره شده است) همبستگی بین حرف آموزشی و تصویر γ محاسبه می‌شود. همبستگی در واقع نشان‌دهنده‌ی میزان شباهت دو تصویر به هم است.

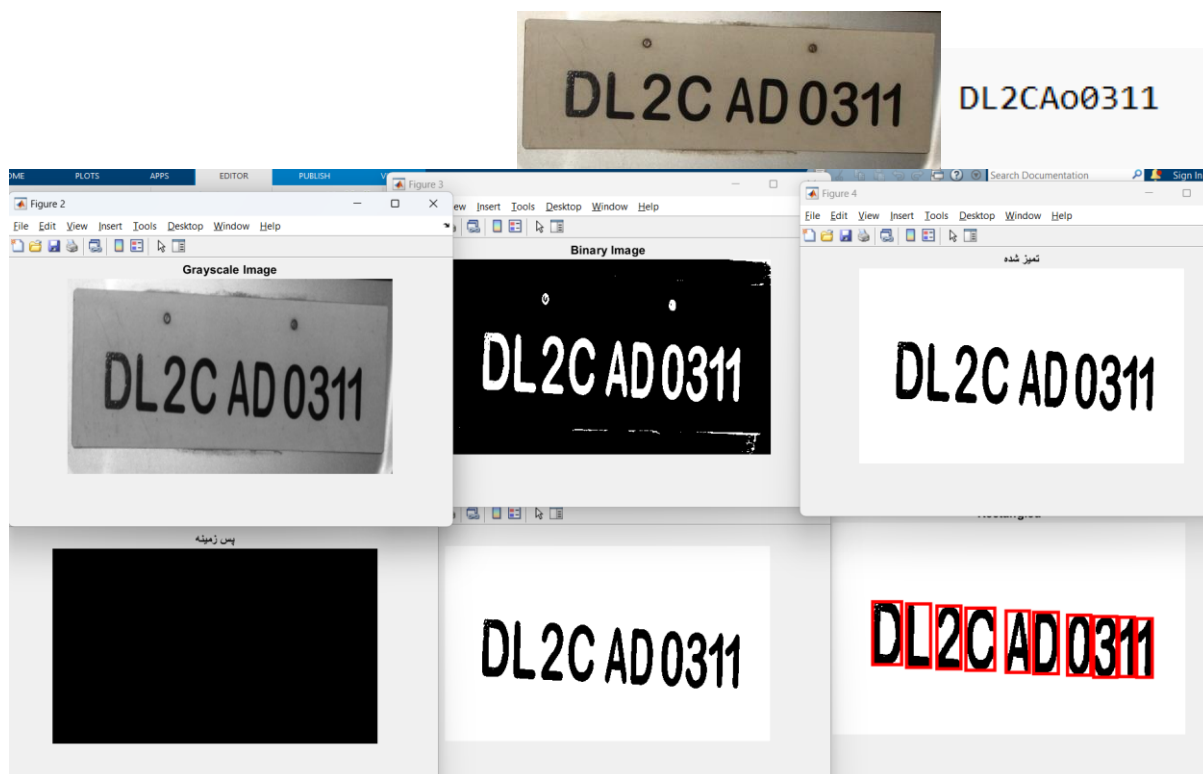
اکنون کد بالاترین مقدار همبستگی را در `ro` پیدا می‌کند. این به ما می‌گوید که تصویر جدید چقدر به یک حرف در مجموعه آموزشی (حروف و اعداد پلاک) شباهت دارد. اگر مقدار بالاترین همبستگی (که در `MAXRO` ذخیره شده) بیشتر از ۰.۴۵ باشد (یعنی شباهت قابل قبول است) کد با استفاده از موقعیتی که همبستگی حداکثر به دست آمد، حرف

متناظر را از مجموعه آموزشی استخراج می‌کند. سپس این حرف را به خروجی نهایی (final_output) اضافه می‌کند.

—۸

سپس یک فایل به نام number_Plate.txt ساخته می‌شود و در حالت نوشتن final_output را به صورت رشته نمایش داده می‌شود.

نمایش ورودی ها و خروجی ها و figure ها:

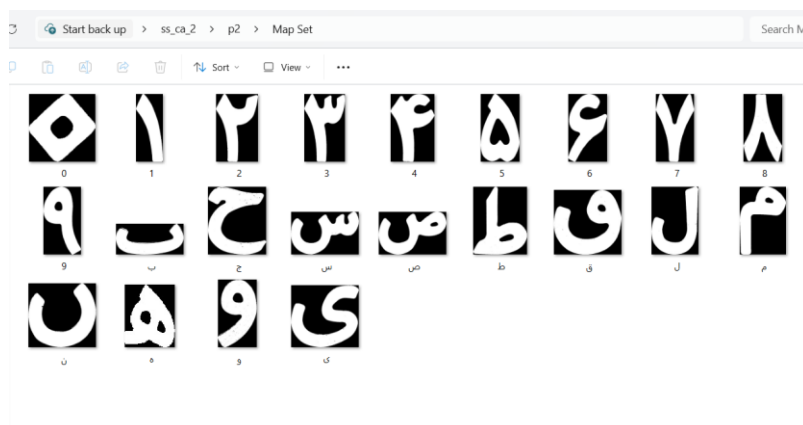


بخش دوم:

تمام توضیحات بخش قبل برای این بخش صادق است و با کمک فایل‌های آپلود شده ، `mapset_loading` در `main_plate` این بخش نیز به راحتی کامل می‌شود.

به همین دلیل صرفاً بخش‌هایی که نیازمند توضیحات بیشتر هستند در این بخش ارائه خواهند شد.

مپ ست:



لود کردن مپ ست فارسی در فایل مجزا از بخش قبل:

```
mapset2trainingset.m  p2.m  mapset2trainingsetFA.m *
clc;clear;close all;clearvars;
cd('C:\Users\alimi\Desktop\ss_ca_2');

directory = dir('./p2/Map Set/');
nam = {directory(3:end).name};
len = length(nam);

TRAINF = cell(2, len);

for i = 1:len
    currentFile = fullfile('./p2/Map Set/', nam{i});
    if isfile(currentFile)
        TRAINF(1, i) = {imread(currentFile)};
        TRAINF(2, i) = {nam{i}(1)};
    else
        warning('File does not exist: %s', currentFile);
    end
end

save('TRAININGSET2.mat', 'TRAINF');
```

به دلیل متفاوت بودن مقادیر ریسایز برای عکس پلاک و مپست `threshold` نیز متفاوت خواهد بود که در کد اعمال شده.

از توابع آماده که در کد آپلود شده در ایلرن قرار داشته استفاده شده. به دلیل هم سایز نبودن اعداد و حروف (برخلاف قسمت قبل) هنگام کورلیشن‌گیری این کار انجام داده می‌شود.

```

62     for k = 1:totalLetters
63         trainingImage = imresize(TRAINF{1, k}, [100, 80]);
64         ro(k) = corr2(trainingImage, Y);
65         disp(ro(k));
66     end

```

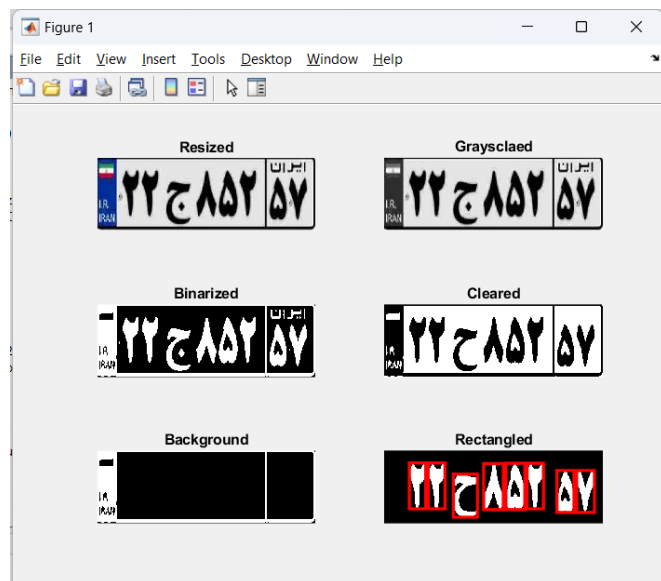
درهنگام تست کردن پلاک‌های مختلف فارسی متوجه شدم در بعضی از پلاک‌ها عدد ۲ اشتباه ۳ شناسایی می‌شود، بنابراین برای رفع این مشکل، اگر اختلاف کورلیشن عدد ۲ و ۳ کمتر از ۰.۱ بود، قسمت مربوطه به عنوان ۲ شناسایی می‌شود. (اگر کورلیشن ۳ بیشینه باشد)

```

67     if abs(ro(3)-ro(4))<0.1
68         ro(3)=ro(3)+0.1;
69     end
70     [MAXRO, pos] = max(ro);
71     if MAXRO > 0.45
72         out = cell2mat(TRAINF(2, pos));
73         final_output = [final_output out];
74     end
75 end

```

نمایش ورودی‌ها و خروجی‌ها و figure ها:



توجه فرمایید که برای خواندن پلاک ۲ رقم چپ (که کوچکتر هستند) دو رقم چپ پلاک، سپس حرف، و بعد از آن ۵ رقم باقی‌مانده خوانده می‌شود.

بخش سوم:

توضیح خط به خط این بخش:

۱- آماده سازی و بارگذاری تصویر

clear all تمامی متغیرهای موجود در فضای کار MATLAB را پاک می کند.

uigetfile این تابع یک پنجره باز می کند تا کاربر بتواند تصویری از نوع JPEG ، BMP ، PNG یا TIFF انتخاب کند.

temp = [path, file]; مسیر کامل تصویر انتخاب شده را ایجاد می کند.

picture = imread(temp); تصویر را با استفاده از مسیر انتخاب شده بارگذاری می کند و آن را در متغیر picture ذخیره می کند.

۲- بارگذاری و تنظیم اندازه تصویر پرچم

flag_template = imread('..\p3\FLAG_TEMP.png'); استفاده می شود) را بارگذاری می کند. (که به عنوان الگو برای شناسایی

picture = imresize(picture, [800 NaN]); تصویر اصل را طوری تغییر اندازه می دهد که ارتفاع آن ۸۰۰ پیکسل باشد و عرضش به تناسب تغییر کند.

figure; یک پنجره جدید برای نمایش تصویر باز می کند.

imshow(picture); تصویر ریسایز شده را نمایش می دهد.

title عنوانی برای عکس تعیین می شود.

۳- تعیین اندازه پرچم

با استفاده از این دستور، عرض و ارتفاع تصویر پرچم (flag_template) استخراج می شود و در width_flag_template و height_flag_template ذخیره می شود.

۴- محاسبه کورلیشن هر کانال (به دلیل اینکه XCORR فقط یک کانال را کورلیشن می گیرد) و ترکیب کورلیشن ها و پیدا کردن مکان پرچم داخل عکس به کمک پیدا کردن بیشینه کورلیشن

normxcorr2 یک تابع است که کورلیشن دو تصویر را محاسبه می کند. در اینجا به طور جداگانه برای هر کانال رنگی (قرمز، سبز، آبی) انجام می شود.

در اینجا کورلیشن ها را با وزن دهی متفاوت ترکیب می کند (نصف وزن برای کانال آبی و ۲۵ درصد برای قرمز و سبز) تا تصویر نهایی کورلیشن کلی به دست آید. (به دلیل اینکه بیشتر پرچم آبی است بیشتر وزن دهی شده)

`max(corr(:))` حداکثر مقدار همبستگی و موقعیت آن را در آرایه پیدا می‌کند.

`ind2sub` اندیس حداکثری را به ردیف و ستون آن در تصویر تبدیل می‌کند.

۶- نمایش تصویر با مستطیل دور پرچم شناسایی شده

یک پنجره جدید برای نمایش تصویر باز می‌کنیم.

سپس تصویر اصلی را دوباره نمایش می‌دهد.

مختصات حداکثر کورلیشن به دست آمده را برای ترسیم مستطیلی که دور پرچم شناسایی شده قرار گیرد، استفاده می‌کند.

با دستور رکت‌انگل مستطیلی زرد دور پرچم رسم می‌شود.

۷- آماده‌سازی و نمایش ناحیه پلاک، استخراج و نمایش تصویر پلاک

در چند خط کد بعد با توجه به اسکیل متفاوت پلاک به کمک عرض تصویر اصلی که ریسایز شده (۸۰۰) و ارتفاع پرچم، و افزودن مقادیر ثابتی مکان پلاک شناسایی می‌شود.

با استفاده از `imcrop`، ناحیه مشخص شده (پلاک) از تصویر اصلی جدا می‌شود و در متغیر `plate` ذخیره می‌شود.

پس از آن مراحل مانند بخش دوم می‌باشد.

نمایش ورودی ها و خروجی ها و figure ها:



۵۹۸۴۴۱۰ ی

بخش چهارم:

متاسفانه این بخش به دلیل گیر کردن در حلقه، ناقص ماند.

فایل ناقص آپلود شد.

الگوریتمی که سعی می‌کردم با آن سوال را حل کنم تقدیم می‌گردد:

خواندن دو فریم ویدئو: الگوریتم فریم‌های مشخصی را از ویدئو بارگذاری می‌کند.

شناسایی پلاک: یک تصویر الگوی پلاک (همان پرچم) بارگذاری و اندازه آن به ابعاد مختلف تغییر می‌یابد. سپس برای هر ابعاد، همبستگی بین ناحیه‌ای از فریم و تصویر الگو محاسبه می‌شود. (کد در اینجا ناقص ماند).

ایجاد تصویر پلاک: با استفاده از بهترین کورلیشن، ناحیه پلاک در هر دو فریم شناسایی و استخراج می‌شود.

محاسبه مسافت: اختلاف موقعیت پلاک‌ها در دو فریم به عنوان مسافت طی شده در نظر گرفته می‌شود.

محاسبه سرعت: با استفاده از مسافت و زمان بین دو فریم، سرعت خودرو محاسبه و نتایج در کنسول نمایش داده می‌شود.



